Na czym polega normalizacja bazy danych i w jakim celu się ją stosuje?

Normalizacja to technika projektowania bazy danych tak, żeby uniknąć redundancji lub anomalii i zminimalizować niebezpieczeństwo rozspójnienia danych. Mówiąc inaczej, normalizacja polega na dostosowaniu schematu relacji, czyli struktury tabel tak, żeby spełniały założenia postaci normalnej.

Postaci normalne

- Pierwsza postać normalna (1NF) – atrybuty są atomowe (nie listy), nie występują powtórzenia.
- Druga postać normalna (2NF) – 1NF + nieklucze nie są częściowo zależne od klucza.

$$\mathrm{K} = (\mathrm{A}, \mathrm{B}),\ \ \mathrm{A} \to \mathrm{C} \quad \times$$

- Trzecia postać normalna (3NF) – 2NF + atrybuty są zależne bezpośrednio od każdego klucza.

$$\mathrm{K} = \mathrm{A},\ \ \mathrm{A} \to \mathrm{B},\ \ \mathrm{B} \to \mathrm{C} \quad \times$$

- Postać normalna Boyce'a Codda (BCNF) – 2NF + każdy atrybut, od którego zależny jest inny atrybut, jest superkluczem relacji.

$$\mathrm{K} = \mathrm{A},\ \ \mathrm{B} \to \mathrm{A} \quad \times$$

- Czwarta postać normalna (4NF) – BCNF + każda zależność wielowartościowa* ma po lewej stronie superklucz.

$$\mathrm{T}(\mathrm{A}, \mathrm{B}, \mathrm{C}),\ \ \mathrm{K} = \mathrm{A},\ \ \mathrm{A} \to \mathrm{B} \perp \mathrm{A} \to \mathrm{C} \quad \longrightarrow \quad \mathrm{T1}(\mathrm{A}, \mathrm{B}),\ \ \mathrm{T2}(\mathrm{A}, \mathrm{C})$$

* Jeśli $\mathcal{R} = \{X,\ Y,\ Z\}$ i $Z \cap (X \cup Y) = \varnothing$, to $\mathcal{R}$ spełnia zależność wielowartościową $X \twoheadrightarrow Y$, gdy dla dowolnych $t_1,\ t_2$ jeśli $t_1[X] = t_2[X]$, to istnieje taka krotka t_3, że $t_1[X, Y] = t_3[X, Y]$ i $t_2[Z] = t_3[Z]$.

Zyski:

- mniejsza ilości danych bez straty informacji
- uniknięcie anomalii przy dodawaniu, aktualizacji, usuwaniu danych
- łatwiejsze tworzenie indeksów, więc przyspieszenie niektórych zapytań
- szybsze zarządzanie transakcjami
- bardziej efektywne składowanie tabel na dysku

Mechanizmy zapobiegające utracie danych w przypadku wystąpienia awarii.

Jednym z zadań SZBD jest zapewnianie tego, żeby zmiany dokonane przez widoczną lub zatwierdzoną transakcję były trwałe, a efekty transakcji wycofanej lub przerwanej przez awarię nie.

- **awaria systemu:** awaria sprzętu, błędy oprogramowania, uszkodzenie dysku
- **błędy logiczne:** np. naruszenie ograniczeń integralnościowych
- **błędy systemowe:** np. zakleszczenia

Do poradzenia sobie z pierwszym rodzajem wystarczy replikacja bazy i rozproszenie danych. W przypadku pozostałych dwóch trzeba odtworzyć bazę po awarii. Przy odtwarzaniu bazy serwer musi być wyłączony dla użytkowników, więc trzeba zrobić to jak najszybciej. Strategie są dwuczęściowe:

1. zbieranie informacji potrzebnych do odtworzenia stanu bazy na bieżąco podczas zwykłych operacji
2. czynności po awarii przywracające spójność, atomowość i trwałości

Strategie SZBD

Brudne strony (zmieniane po ostatnim zapisie na dysku) mogą być w buforze długo po zatwierdzeniu transakcji. Jeśli nastąpi awaria, to zmiany są tracone.
Żeby przywrócić poprawny stan bazy można wykonać:

- UNDO, wycofanie zmian – zapewnia atomowość,
- REDO, powtórzenie transakcji – zapewnia trwałość.

Są różne strategie SZBD:
Czy niezatwierdzona transakcja może nadpisać wartość w pamięci trwałej?

- steal: TAK
- no-steal: NIE

Czy wszystkie zmiany muszą być zapisane w pamięci trwałej przed zatwierdzeniem transakcji?

- force: TAK
- no-force: NIE

Przy **no-steal + force** nie trzeba wykonywać UNDO i REDO, ale jest mało efektywna i istnieje ryzyko samozakleszczenia transakcji, jeśli braknie miejsca w buforze.
Częściej wykorzystywana jest **steal + no-force** opakowana przez **protokół WAL (write-ahead log)**. Przechowuje się dziennik z logami zmian jeszcze nie zapisanych na dysku. Jest wystarczający, żeby odzyskać zawartość bazy za pomocą UNDO i REDO.
Żeby dzienniki nie rozrastały się za bardzo, tworzy się punkty kontrolne. W przypadku awarii powtarzane są tylko transakcja zatwierdzona po ostatnim punkcie kontrolnym, a niezatwierdzone są wycofywane.

Algorytm ARIES

Algorytm ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) służy do odtwarzania transakcji, korzystając z dziennika WAL i blokad. Składa się z trzech faz:

1. **Analiza**: rekonstrukcja Tabeli Brudnych Stron (DPT) i Tabeli Transakcji (TT), wyznaczenie pierwszego wpisu o zmianach tworzących brudną stronę.
2. **Faza REDO**: powtórzenie operacji od powstania brudnej strony, przywrócenie stanu sprzed awarii.
3. **Faza UNDO**: wycofanie niezatwierdzonych transakcji, dodanie wpisów kompensacyjnych do dziennika.

Podstawowe techniki w optymalizacji zapytań w bazach danych.

Schemat przetwarzania przez SZBD zapytań SQL na kod wykonywalny:

1. **zapytanie SQL**
   ↓ tłumaczenie kanoniczne
2. **wyrażenie algebry relacyjnej**
   ↓ optymalizacja heurystyczna
3. **dostosowane wyrażenie**
   ↓ optymalizacja na podstawie kosztu
4. **plan fizyczny**
   ↓ dodanie instrukcji wykonania operatorów
5. **kod wykonywalny**

Optymalizacja heurystyczna

iteracyjna modyfikacja wyrażenia kanonicznego, tak żeby uzyskać jak najlepszy plan logiczny, nie zmieniając wyniku.

- selekcje jak najwcześniej – zmniejsza to rozmiar relacji
- najbardziej restrykcyjna selekcja jak najwcześniej (z łączności selekcji)
- iloczyn kartezjański + selekcja (w takiej kolejności) $\longrightarrow$ złączenie
- rzutowania jak najwcześniej – zmniejsza to rozmiar relacji
- niepowtarzanie obliczeń – jeśli jakieś wyrażenie powtarza się w drzewie, zapamiętaj jego wynik

Optymalizacja oparta na podstawie kosztów

wybór „najtańszego” planu na podstawie szacowania kosztu dla podzbioru potencjalnych planów.

Przybliżony koszt planu:

- szacunkowy koszt każdego operatora (zależy od rozmiarów argumentów)
- szacunkowy rozmiar wyniku każdego operatora

System bazodanowy przechowuje statystyki dotyczące relacji i atrybutów.

Próbkowanie

szacowanie selektywności na podstawie próbek. Próbki są aktualizowane po większych zmianach w tabeli.

Czym są indeksy w bazach danych? Kiedy warto je tworzyć? Jakie struktury danych są najczęściej wykorzystywane jako indeksy?

Indeks to struktura danych na adresy fizyczne krotek i atrybuty, po których sortowane są krotki (klucze wyszukiwania $\neq$ klucze relacji) do szybkiego wyszukiwania, dodawania i usuwania.

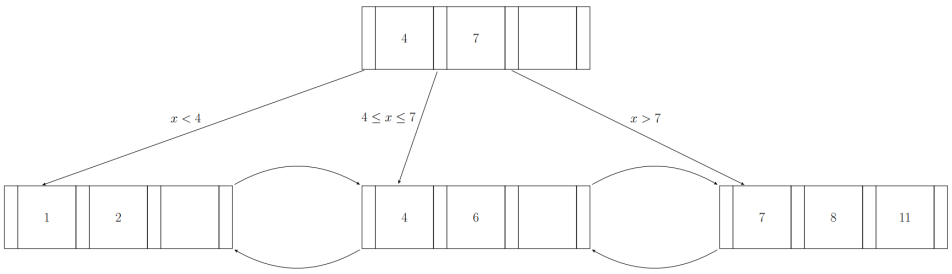
0.1 Implementacje indeksów

B+-drzewo

umożliwia wyszukiwanie, dostęp sekwencyjny, wstawianie i usuwanie w $\mathcal{O}(\log_M N)$.

- ukorzenione, zbalansowane drzewo M-arne,
- każdy wierzchołek poza korzeniem ma k kluczy, gdzie $\frac{M}{2} - 1 \leq k \leq M - 1$,
- węzeł wewnętrzny o k kluczach ma $k + 1$ potomków,
- węzły wewnętrzne przechowują tablice z parami <node*, key>, a liście wartości (krotki lub adresy) <value, key>.

Ogólnie im wolniejsza pamięć, tym większy jest optymalny rozmiar węzła. Najszybciej można zbudować B+-drzewo, sortując klucze, a potem budując indeks od dołu (*bulk loading*). Zalety:



- mała liczba odczytów z dysku,
- efektywne na każdym poziomie hierarchii pamięci,
- względnie efektywne i dla dostępu swobodnego, i sekwencyjnego,
- efektywne dla różnych typów danych.

Wada: rozmiar rośnie liniowo

Indeks pokrywający

przechowuje też wartości niektórych atrybutów innych niż klucz wyszukiwania.

Indeks haszowy

W przeciwieństwie do B+-drzew, rozmiar indeksów haszowych rośnie skokowo, gdy przydzielana jest dodatkowa pamięć dla nowych kubełków. Rozmiar nie jest zależny od rozmiaru haszowanych wartości.

Indeks bitmapowy

koduje w mapach bitowych wartości krotek z wybranej kolumny. Pozwala sprowadzić niektóre zapytania do operacji bitowych.

Kiedy warto korzystać z indeksów?

Domyślnie dla każdej tabeli tworzony jest indeks odpowiadający kluczowi głównemu i każdemu unikalnemu atrybutowi.
Im więcej indeksów, tym szybsze wyszukiwanie, ale też tym więcej miejsca zajmuje baza, a czas synchronizacji i planowania zapytania jest dłuższy.

- indeks zgrupowany (dane w kolejności jak na stronach):
  - do operacji na zakresach
  - do pobierania danych w określonym porządku
  - do zapytań na wielu kolumnach

  Efektywne, kiedy wartości są krótkie, rzadko się powtarzają i są rzadko zmieniane.
- indeks częściowy (na wybranych wartościach atrybutu)
  - do zapytań na podzbiorze krotek
- indeks haszowany
  - do zapytań równościowych
- indeks bitmapowy
  - dla atrybutów o niewielkiej dziedzinie
- indeks B+-drzewa
  - do zapytań równościowych i na zakresach

Indeksy mogą działać wolniej, gdy zapytanie wymaga przeskanowania dużej części tabeli lub dane są często zmieniane.

Pojęcie kluczy w teorii relacyjnych baz danych.

Klucz – minimalny w sensie inkluzji pozbiór atrybutów $\{B_1, \ldots, B_k\} \subseteq \{A_1, \ldots, A_n\}$ relacji $\mathcal{R}(A_1, \ldots, A_n)$, który jest wystarczający do rozróżnienia krotek, czyli:

$$t_1[B_1, \ldots, B_k] = t_2[B_1, \ldots, B_k] \implies t_1 = t_2$$

Klucz podstawowy – jeden wybrany klucz dla danej relacji. Jej pozostałe klucze, to **klucze wtórne (kandydujące)**.

Klucz obcy relacji – zbiór jej atrybutów, który jest kluczem innej relacji. Atrybuty muszą mieć identyczną dziedzinę w obu relacjach i dla każdej krotki istnieje co najwyżej jedna taka krotka w drugiej relacji, że ich wartości są równe na kluczu.

Omów pojęcie transakcji i przedstaw jej własności.

Transakcja to jedna lub wiele operacji czytania i pisania z porządkiem wykonania traktowana jako całość. Transakcje są wykonane wszystkie albo żadna (abort / rollback).

Żeby zrównoleglenie nie przynosiło więcej problemów niż pożytku, trzeba zachować własności ACID:

- **Atomowość (*atomicity*)** – transakcja jest niepodzielna, czyli nie może być wykonana częściowo. Można logować zmiany lub tworzyć kopie zmienianych stron plików i umożliwiać odczyt dopiero po zatwierdzeniu (*shadow paging*).

- **Integralność (*consistency*)** – po zatwierdzeniu transakcji baza danych musi spełniać wszystkie warunki poprawności
  Pojedyncze operacje mogą tymczasowo naruszać warunki, ale po zakończeniu transakcji musi być przywrócony poprawny stan.
- **Izolacja (*isolation*)** – wynik równoległego wykonania transakcji musi być szeregowalny
  Dwie transakcje wykonywane w tym samym czasie nie mogą wpływać na siebie. Są metody pesymistyczne, które nie pozwalają na zaistnienie problemu, i optymistyczne, które naprawiają to, co się zepsuło.
- **Trwałość (*durability*)** – efekty pomyślnie zakończonej transakcji są trwałe w bazie danych
  Wyniki zatwierdzonej transakcji nie są tracone nawet w przypadku twardych awarii (brak zasilania, błąd oprogramowania). Dane mogą zniknąć z pamięci głównej, ale pozostają zapisane na dysku. Można zapisywać zmiany na dysku w trakcie transakcji lub przechowywać wystarczające informacji, żeby odtworzyć stan sprzed awarii.

Modelowanie baz danych za pomocą związków encji.

Encja (*entity*) to obiekt przechowywany w bazie. Zbiór encji to zbiór obiektów posiadających te same atrybuty, które mogą być:

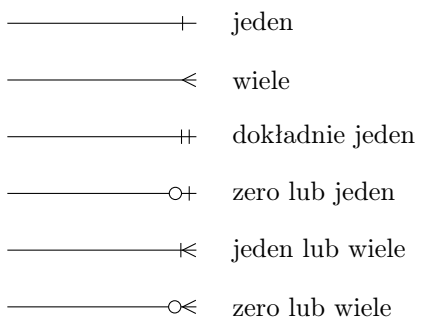
- proste (o niepodzielnych wartościach) lub złożone (np. adres),
- jednowartościowe (np. id) lub wielowartościowe (np. adresy mailowe danej osoby),
- pochodne, których wartość jest określona pewną funkcją na wartościach innych atrybutów.

Instancje encji są parami różne, więc dla zbioru encji pojęcia superklucza, klucza, klucza głównego i klucza wtórnego działają analogicznie jak w modelu relacyjnym.

Słaba encja to taka encja, która występuje tylko w kontekście innych encji (np. instytut w kontekście wydziału). Słabe encje nie mają własnego identyfikatora, a ich klucz główny zależy od klucza encji, przy których występują.

Związek (*relationship*) to wzajemna zależność pomiędzy dwiema lub więcej encjami. Instancja związku reprezentuje związek między instancjami encji. Związek może być:

- unarny, binarny, ternarny, n-arny (klasyfikacja ze względu na stopień)
- opcjonalny, obowiązkowy (klasyfikacja ze względu na klasę przynależności)
- 1 : 1, 1 : M, M : M (klasyfikacja ze względu na typ asocjacyjny)

Superkluczem dla $\mathcal{R}$ zbioru związków o atrybutach $A_1, \ldots, A_k$ pomiędzy zbiorami encji $E_1, \ldots, E_n$ o kluczach głównych $\mathrm{PK}(E_1), \ldots, \mathrm{PK}(E_n)$ jest $\mathrm{PK}(E_1) \cup \ldots \cup \mathrm{PK}(E_n) \cup \{A_1, \ldots, A_k\}$.

Do reprezentacji modelu związków encji za pomocą notacji kurzej stopki.